

Names: Dave Burgoin, Moyo Ogunjobi, Jacob Plourde

Group #: 6

Course: ENSF 460 - Embedded Software and Hardware Systems

Assignment Number: Assignment 1 Driver Project

Date Submitted: September 25, 2026

1 Peripheral Initialization

All GPIO pin configuration is performed directly after entering the `main()` function and before entering the `while(1)` super loop. This initialization configures the required pins as either inputs for the three push buttons or an output for the LED.

```
1  int main()
2  {
3      AD1PCFG = 0xFFFF;    // Configure analog-capable pins as digital I/O
4      TRISB = 0x90;        // Configure RB4 and RB7 as inputs; remaining PORTB pins
                             // as outputs
5      TRISA = 0x10;        // Configure RA4 as input; remaining PORTA pins as
                             // outputs
6
7      CNPU1 = CNPU2 = 0x0; // Disable internal pull-up resistors
8      CNPD1 = 0x3;        // Enable pull-down resistors for RB4 and RA4
9      CNPD2 = 0x80;        // Enable pull-down resistor for RB7
10
11     ...
```

2 Super Loop and Program Logic

The main program logic is contained within the `while(1)` super loop. The three push buttons are read using three `#define` macros. Each macro reads the appropriate `PORTx` register, applies a bit mask to isolate the required input bit, and performs a Boolean comparison using `!= 0`.

The resulting Boolean values are shifted into their corresponding bit positions to form the control value used by the switch statement, shown in Table 1. The switch statement then determines the required LED state and selects the appropriate delay value based on the current combination of push-button inputs. Functionally, this switch statement is a complete state machine, in which one of the states is always active on each iteration of the super loop, and each state can directly switch into any other state.

PB3	PB2	PB1	Switch Control Value
0	0	0	0
0	0	1	1
0	1	0	2
0	1	1	3
1	0	0	4
1	0	1	5
1	1	0	6
1	1	1	7

Table 1: Push-Button State Bitmasks

```
1  // Combine the three button inputs into a 3-bit control value: PB3|PB2|PB1
2  switch (PB3|PB2|PB1)
3  {
```

```

4  case 1:                                // PB1 only: blink every 750 ms
5      LED_TOGGLE;
6      blink_delay = 750;
7      break;
8
9  case 2:                                // PB2 only: blink every 2000 ms
10     LED_TOGGLE;
11     blink_delay = 2000;
12     break;
13
14  case 4:                                // PB3 only: blink every 5000 ms
15     LED_TOGGLE;
16     blink_delay = 5000;
17     break;
18
19  case 3:                                // PB1 + PB2
20  case 5:                                // PB1 + PB3
21  case 6:                                // PB2 + PB3
22  case 7:                                // All three buttons
23     LED_ON;                            // Multiple buttons pressed: keep LED
        continuously on
24     blink_delay = 0;                    // No delay required while LED is continuously on
25     break;
26
27  case 0:                                // No buttons pressed
28  default:                              // Safe default for any unexpected control value
29     LED_OFF;                            // Keep LED off
30     blink_delay = 0;                    // No delay required while LED is off
31     break;
32 }

```

3 Datatype Choices

In general, integer variables are declared as `uint16_t` or `int16_t` because the PIC24F16KA101 is a 16-bit microcontroller, and so it has hardware native support for 16-bit integers. Using fixed-width integer types also makes the intended size of each variable explicit.

The datatype used to count for loop iterations in our `delay_ms()` function is `uint16_t`. However, the maximum value for `uint16_t` is 65,535, which would only be able to delay for less than 150ms. A 5-second delay, for example, requires a value of approximately 2,500,000, which is far out of the `uint16_t` range. Therefore, we use a doubly-nested for loop, with the inner one counting a single millisecond, and the outer one counting the number of milliseconds. Alternatively, a `uint32_t` can be used, but it does not have native hardware support, so it will be much less efficient. The ASM shows that a `uint32_t` based for loop is 37 instructions, whereas a nested `uint16_t` loop only takes 24 instructions, with a much more efficient execution.

```

1  #define DELAY_1ms 501
2
3  void delay_16(uint16_t ms) {
4      for (uint16_t i = 0; i < ms; i++) {
5          for (volatile uint16_t j = 0; j < DELAY_1ms; j++) (void)0;
6      }
7  }
8
9  void delay_32(uint32_t ms) {
10     for(volatile uint32_t i = 0; i < ms * DELAY_1ms; i++) (void)0;
11 }

```

3.1 delay_16 Assembly

```

1  000002ce <_delay_16>:
2      2ce: 04 00 fa      lnk      #0x4
3      2d0: 00 00 eb      clr.w     w0
4      2d2: 00 0f 78      mov.w     w0, [w14]
5      2d4: 0b 00 37      bra      0x2ec <.L2>
6      //Outer Loop - K iterations
7  000002d6 <.L5>:
8      2d6: 00 00 eb      clr.w     w0
9      2d8: 10 07 98      mov.w     w0, [w14+2]
10     2da: 03 00 37      bra      0x2e2 <.L3>
11     //Inner Loop - N iterations
12  000002dc <.L4>:
13     2dc: 1e 00 90      mov.w     [w14+2], w0
14     2de: 00 00 e8      inc.w     w0, w0
15     2e0: 10 07 98      mov.w     w0, [w14+2]
16  000002e2 <.L3>:
17     2e2: 9e 00 90      mov.w     [w14+2], w1
18     2e4: 40 1f 20      mov.w     #0x1f4, w0
19     2e6: 80 8f 50      sub.w     w1, w0, [w15]
20     2e8: f9 ff 36      bra      LEU, 0x2dc <.L4>
21     //Inner loop Ends
22     2ea: 1e 0f e8      inc.w     [w14], [w14]
23  000002ec <.L2>:
24     2ec: 00 40 80      mov.w     0x800, w0
25     2ee: 9e 00 78      mov.w     [w14], w1
26     2f0: 80 8f 50      sub.w     w1, w0, [w15]
27     2f2: f1 ff 39      bra      NC, 0x2d6 <.L5>
28     //Outer Loop Ends
29     2f4: 00 80 fa      ulnk
30     2f6: 00 00 06      return

```

Total Complexity $N(7) + K(14)$ assuming N of 10 and K of 10 where we call N, K times
 $N(10) \times 10 + K(10) = 840$ instructions

3.2 delay_32 Assembly

1	000002f8	<_delay_32>:		
2	2f8: 04 00 fa	lnk	#0x4	
3	2fa: 60 00 b8	mul.uu	w0, #0x0, w0	
4	2fc: 00 8f be	mov.d	w0, [w14]	
5	2fe: 04 00 37	bra	0x308 <.L7>	
6	//Loop Starts			
7	00000300	<.L8>:		
8	300: 12 00 20	mov.w	#0x1, w2	
9	302: 03 00 20	mov.w	#0x0, w3	
10	304: 1e 0f 41	add.w	w2, [w14], [w14]	
11	306: 5e 97 49	addc.w	w3, [++w14], [w14--]	
12	00000308	<.L7>:		
13	308: 00 40 80	mov.w	0x800, w0	
14	30a: 80 00 eb	clr.w	w1	
15	30c: f2 14 20	mov.w	#0x14f, w2	
16	30e: 02 89 b9	mul.ss	w1, w2, w2	
17	310: 02 01 78	mov.w	w2, w2	
18	312: 60 02 b9	mul.su	w0, #0x0, w4	
19	314: 84 01 78	mov.w	w4, w3	
20	316: 03 01 41	add.w	w2, w3, w2	
21	318: f3 14 20	mov.w	#0x14f, w3	
22	31a: 03 00 b8	mul.uu	w0, w3, w0	
23	31c: 01 01 41	add.w	w2, w1, w2	
24	31e: 82 00 78	mov.w	w2, w1	
25	320: be 0f 50	sub.w	w0, [w14++], [w15]	
26	322: ae 8f 58	subb.w	w1, [w14--], [w15]	
27	324: ed ff 3e	bra	GTU, 0x300 <.L8>	
28	//Loop Ends			
29	326: 00 80 fa	unlk		
30	328: 00 00 06	return		

Total Complexity $N(19)$ assuming N of $N(19) \times 100 = 1900$ instructions

4 Blocking Delay and LED Response

The LED does not turn on or off immediately after a push button is pressed or released because the current delay implementation is blocking. The program enters a for loop and remains inside that loop until the delay count has been reached.

While the program is executing the delay loop, the push-button inputs are not being checked. The CPU must first finish the current delay before execution can continue through the `while(1)` super loop. Once the delay finishes, the push buttons are read again, the switch statement determines the new state, and the LED state and delay value are updated.

Therefore, the maximum response time to a button change depends on where the program currently is within its delay. This behaviour is a consequence of using a simple blocking software delay rather than a non-blocking/interrupt based timer implementation.